

AI Application / Agent Portfolio Summary

프로젝트 요약본 - 공개 제출 및 면접 사전 공유용

핵심 포지셔닝

LLM/RAG Agent, 시계열 파운데이션 모델, AI-OCR, MCP Tooling, Hybrid Symbolic RL PoC를 수행하며, AI 모델을 검증 가능하고 추적 가능한 서비스 구조 안에 배치하는 데 집중했습니다.

AI Coding Assistant 활용

AI Coding Assistant는 Skill과 프로젝트 지침을 통해 요구사항, 설계 문서, 구현 계획 문서를 실제 구현 작업에 반영하도록 지시하는 방식으로 활용했습니다. 구현된 코드는 테스트 코드로 1차 확인하고, 프로그램 실행 중 발생한 에러 로그와 실패 원인을 다시 분석해 구현 오류를 수정하는 반복 과정에 사용했습니다. 즉, 단순 코드 생성 도구가 아니라 문서 기반 구현, 테스트 피드백, 실행 오류 분석, 수정 사이클을 빠르게 반복하기 위한 개발 보조 도구로 활용했습니다.

Portfolio Overview

Project	핵심 가치	운영화 시 핵심 보강
Self-Correcting RAG Service PoC	검색-생성-검증-재생성 단계를 분리하여 RAG 응답의 환각 가능성을 줄이는 Agentic RAG 구조를 검토했습니다. 핵심은 LLM 답변을 바로 반환하지 않고, 근거 적합성과 누락 여부를 점검한 뒤 필요하면 재검색 또는 재생성으로 되돌리는 흐름입니다.	정답/평가 질문셋 구축 / 답변-근거 문장 alignment 저장 / groundedness/faithfulness 자동 평가
Time Series AI Platform PoC	Moirai, Chronos 등 시계열 파운데이션 모델과 TimeCopilot 파이썬 패키지를 활용해 예측, 평가, 시각화, LLM 기반 분석을 연결하는 플랫폼 구조를 검토했습니다. 대형 모델 추론 병목도 함께 분석해 실행 파이프라인 개선 방향을 정리했습니다.	MLflow 기반 실행 추적 / 모델별 benchmark 자동화 / GPU memory profiling
AI-OCR Pipeline PoC	PaddleOCR와 VLM을 결합해 문서 구조화 추출과 원문 근거 추적을 함께 수행하는 AI-OCR 파이프라인을 검토했습니다. OCR line evidence와 VLM의 ocrlids를 연결해 bbox, raw text, confidence를 복원하는 구조가 핵심입니다.	문서 유형별 schema/version 관리 / validation failure dataset 구축 / human review UI
Hybrid Symbolic RL PoC	전력시장 입찰 수익 최적화를 위해 LLM과 강화학습을 결합한 PoC입니다. 초기에는 LLM 코드 생성 + GRPO를 시도했으나, 안정성 한계를 확인하고 LLM은 전략 해석, SAC는 수치 행동 최적화를 맡는 Hybrid Symbolic RL 구조로 전환했습니다.	실제 시장 정산 규칙 반영 / 발전소/ESS 운영 제약 정교화 / ESS degradation cost 반영
MCP RAG Bridge PoC	기존 RAG API를 AI Agent가 표준 MCP 인터페이스로 사용할 수 있도록 FastMCP 기반 브리지 서버로 감싼 프로젝트입니다. Tool, Resource, Prompt를 분리하고, end-to-end 질의와 분해형 query primitive를 함께 제공하는 구조를 정리했습니다.	persistent session store / tool permission scope / tool invocation audit log

1. Self-Correcting RAG Service PoC

중요 요약

검색-생성-검증-재생성 단계를 분리하여 RAG 응답의 환각 가능성을 줄이는 Agentic RAG 구조를 검토했습니다. 핵심은 LLM 답변을 바로 반환하지 않고, 근거 적합성과 누락 여부를 점검한 뒤 필요하면 재검색 또는 재생성으로 되돌리는 흐름입니다.

대표 아키텍처 흐름



Tech Stack

Python	LangGraph	LangChain
FastAPI	Vector DB	LLM API

Core Implementation Logic

- Retriever가 관련 문서 chunk 검색
- Generator가 context 기반 초안 답변 생성
- Self-check node가 groundedness와 누락 여부 판단
- 검증 실패 시 재검색 또는 재생성 수행

Architecture Pattern

- LangGraph node/edge workflow
- Retriever/Generator/Checker 책임 분리
- Self-Correction Loop
- Agentic RAG Pipeline

Result

단순 질의응답형 RAG가 아니라 검증과 교정 루프를 포함한 구조로 정리했습니다. 답변 품질 저하 원인을 검색 실패, context 부족, 생성 오류로 분리해 디버깅 가능한 구조를 만들었습니다.

운영화하려면 추가 진행되어야 하는 것

- 정답/평가 질문셋 구축
- 답변-근거 문장 alignment 저장
- groundedness/faithfulness 자동 평가
- query log와 사용자 feedback loop 운영
- LLM 호출 비용과 latency budget 관리

2. Time Series AI Platform PoC

중요 요약

Moirai, Chronos 등 시계열 파운데이션 모델과 TimeCopilot 파이썬 패키지를 활용해 예측, 평가, 시각화, LLM 기반 분석을 연결하는 플랫폼 구조를 검토했습니다. 대형 모델 추론 병목도 함께 분석해 실행 파이프라인 개선 방향을 정리했습니다.

대표 아키텍처 흐름



Tech Stack

Python	PyTorch	DeepSpeed
Moirai	Chronos	TimeCopilot
Pandas	Matplotlib/Plotly	Streamlit
FastAPI	Redis/RQ	MLflow
A100 GPU		

Core Implementation Logic

- 시계열 입력 데이터를 표준 schema로 정규화
- 모델 alias 기반 forecaster 생성
- Cross-validation과 metric 계산
- 예측 결과 plot artifact 생성
- LLM 기반 자연어 분석 연결
- batch/device/loading 병목 분석

Architecture Pattern

- ForecastPipeline architecture
- Model factory pattern
- Metric/Plot strategy registry
- Runtime cache bundle
- Async job orchestration

Result

모델 분산 자체를 완전 성공했다고 보기는 어렵지만, 데이터 처리와 실행 흐름 최적화를 통해 추론 시간을 줄이는 방향을 확인했습니다. 예측 결과를 그래프와 자연어 설명으로 연결하는 분석 Copilot 가능성을 확인했습니다.

운영화하려면 추가 진행되어야 하는 것

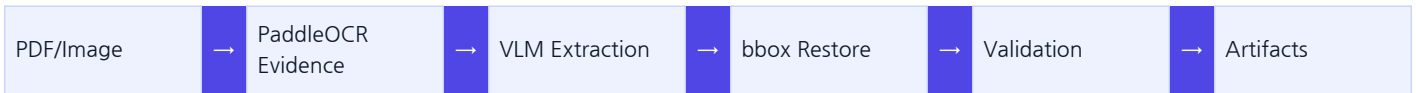
- MLflow 기반 실험 추적
- 모델별 benchmark 자동화
- GPU memory profiling
- 비동기 inference queue 운영
- forecast drift monitoring
- 모델/데이터 버전 관리

3. AI-OCR Pipeline PoC

중요 요약

PaddleOCR와 VLM을 결합해 문서 구조화 추출과 원문 근거 추적을 함께 수행하는 AI-OCR 파이프라인을 검토했습니다. OCR line evidence와 VLM의 ocrIds를 연결해 bbox, raw text, confidence를 복원하는 구조가 핵심입니다.

대표 아키텍처 흐름



Tech Stack

Python	FastAPI	PaddleOCR
PyMuPDF	Pillow/OpenCV	VLM API
SQLAlchemy	SQLite/PostgreSQL	Playwright

Core Implementation Logic

- PDF/Image를 page image로 렌더링
- OCR 전처리와 line-level text/bbox/score 추출
- OCR evidence를 ocrId 기반으로 정규화
- VLM이 structured field와 ocrIds 반환
- ocrIds 기반 bbox/raw/confidence 복원
- total/line item/discount validation 수행
- result/trace/raw artifact 저장

Architecture Pattern

- Evidence-driven extraction
- OCR engine과 VLM extraction layer 분리
- bbox restoration
- Validation rule chain
- Artifact store + DB index 분리
- Human review/ERP integration boundary

Result

VLM의 구조화 능력과 OCR의 좌표/근거 추적 능력을 결합한 구조를 정리했습니다. 추출값의 원문 위치, raw OCR text, validation 결과를 함께 남겨 디버깅 가능한 AI-OCR 파이프라인으로 확장했습니다.

운영화하려면 추가 진행되어야 하는 것

- 문서 유형별 schema/version 관리
- validation failure dataset 구축
- human review UI
- 개인정보 마스킹과 접근 제어
- artifact retention 정책
- OCR/VLM benchmark 자동화
- ERP 연동 재시도/감사 로그

4. Hybrid Symbolic RL PoC

중요 요약

전력시장 입찰 수익 최적화를 위해 LLM과 강화학습을 결합한 PoC입니다. 초기에는 LLM 코드 생성 + GRPO를 시도했으나, 안정성 한계를 확인하고 LLM은 전략 해석, SAC는 수치 행동 최적화를 맡는 Hybrid Symbolic RL 구조로 전환했습니다.

대표 아키텍처 흐름



Tech Stack

Python	PyTorch	Unsloth
TRL	SAC	Gymnasium-style Environment
Pandas	NumPy	Matplotlib
LLM API/Local LLM		

Core Implementation Logic

- LLM이 strategy(state) 코드 생성하는 초기 접근 실험
- syntax/safety/sandbox execution 검증
- GRPO reward function 분리
- LLM Symbolic Agent가 market regime/action guideline 생성
- guideline embedding을 RL state에 주입
- SAC가 DA/RT 수치 action 결정
- requested/executed action mismatch 기록

Architecture Pattern

- Initial GRPO code-generation RL
- Hybrid Symbolic RL
- LLM Symbolic Agent + Numerical Optimizer 역할 분리
- Hierarchical DA/RT SAC
- Two-Settlement Market Environment
- Replay buffer 분리

Result

최종 운영 수준 수익률 검증보다는 구조적 결론이 핵심입니다. LLM이 직접 코드를 생성하거나 수치 action을 결정하는 방식보다, LLM은 해석과 가이드라인을 맡고 RL이 수치 최적화를 맡는 역할 분리가 더 안정적이라는 결론을 도출했습니다.

운영화하려면 추가 진행되어야 하는 것

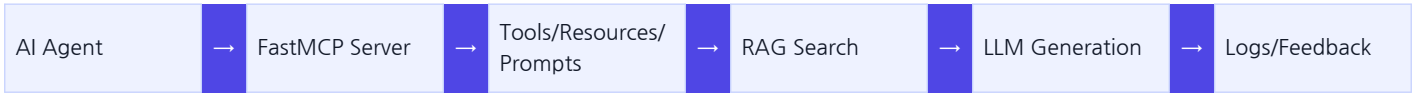
- 실제 시장 정산 규칙 반영
- 발전소/ESS 운영 제약 정교화
- ESS degradation cost 반영
- offline RL evaluation
- simulation-to-real gap 검증
- risk constraint와 입찰 제한 조건 추가
- reward scale normalization

5. MCP RAG Bridge PoC

중요 요약

기존 RAG API를 AI Agent가 표준 MCP 인터페이스로 사용할 수 있도록 FastMCP 기반 브리지 서버로 감싼 프로젝트입니다. Tool, Resource, Prompt를 분리하고, end-to-end 질의와 분해형 query primitive를 함께 제공하는 구조를 정리했습니다.

대표 아키텍처 흐름



Tech Stack

Python	FastMCP	MCP
httpx	Pydantic	Starlette-compatible Routes
Qdrant/pgvector	PostgreSQL	OpenAI-compatible LLM endpoint
SSE/stdio Transport		

Core Implementation Logic

- 기존 RAG API를 MCP Tool로 노출
- ask_rag_agent end-to-end tool 제공
- prepare/plan/search/rerank/generate 분해형 tool 제공
- rag://categories/documents/query-options/logs Resource 제공
- Prompt로 RAG workflow 안내
- thread_id 기반 session state 유지
- FAQ-first workflow와 streaming event 정규화

Architecture Pattern

- Tool/Resource/Prompt catalog pattern
- FastMCP bridge architecture
- Container-based dependency wiring
- Adapter boundary pattern
- QueryCapabilityService + QueryStageService 분리
- Memory/Postgres swappable store
- canonical vs monolith parity structure

Result

기존 API를 단순 wrapper가 아니라 AI Agent가 사용할 수 있는 MCP contract로 재구성했습니다. query log, feedback/FAQ, file metadata, vector search, generation adapter를 교체 가능한 경계로 분리했습니다.

운영화하려면 추가 진행되어야 하는 것

- persistent session store
- tool permission scope
- tool invocation audit log
- multi-tenant isolation
- contract versioning
- prompt/catalog 관리 UI
- evaluation pipeline
- agent별 adapter routing

Operationalization Roadmap

프로젝트별 세부 사항은 다르지만, 운영화 관점에서 공통적으로 필요한 작업은 다음과 같습니다.

영역	추가 진행 사항
품질 평가	프로젝트별 평가셋, regression test, benchmark 자동화
추적성	query log, artifact, error trace, model/input/output versioning
운영 안정성	retry, timeout, queue, monitoring, alert, fallback policy
보안	권한, tenant isolation, 개인정보 마스킹, audit log
비용 관리	LLM token usage, GPU inference cost, storage retention policy
사용자 피드백	human review, feedback/FAQ promotion, failure dataset 구축

요약

이 포트폴리오의 핵심은 특정 모델이나 패키지 사용 자체가 아니라, AI 모델을 실제 서비스 구조에서 검증 가능하고 추적 가능하게 만드는 아키텍처 판단입니다. RAG는 self-check와 로그, OCR은 evidence와 artifact, MCP는 Tool/Resource/Prompt contract, RL은 LLM과 수치 optimizer의 역할 분리, 시계열은 예측-평가-분석 pipeline을 중심으로 정리했습니다.